01. Foundations of Parallel Computing & Performance Measures

This chapter introduces the fundamental concepts, performance metrics, and basic patterns (skeletons) used in parallel computing to evaluate and structure concurrent execution.

1. Performance Metrics

To evaluate the quality and efficiency of a parallel system, we rely on several quantitative metrics:

1.1 Latency (L)

Latency is the time required to complete a single task from its beginning to its end. In a system composed of multiple stages (e.g., a pipeline), it is the sum of the times spent in each individual stage.

1.2 Service Time (T_s)

Service Time is the time interval between the emission of two consecutive results. It dictates the maximum throughput of the system. * In a sequential execution, T_s is equal to the Latency. * In a parallel system (like a pipeline), the overall service time is bottlenecked by the slowest stage: $T_s = \max\{T_{s_1}, T_{s_2}, \ldots, T_{s_k}\}$.

1.3 Completion Time (T_c)

The total time required to process m tasks. Given a pipeline or a generic parallel system, the initial phase involves filling the system (paying the latency L), and subsequently, results are emitted every T_s.

$$T_c = L + (m-1)T_s$$

For a very large number of tasks ($m \gg 1$), the Completion Time can be approximated as $T_c \approx mT_s$.

1.4 Speedup (S_p)

Speedup measures the relative performance gain of executing a program on n parallel processing elements compared to a sequential execution.

$$S_p(n) = \frac{T_{seq}}{T_{par}(n)}$$

Where T_{seq} is the completion time of the best sequential algorithm and $T_{par}(n)$ is the completion time of the parallel algorithm using n workers. * Ideally, we seek **linear speedup** ($S_p(n) = n$). * Speedup is theoretically bounded by the critical path: $S_p \leq \frac{\text{Work}}{\text{Span}}$.

1.5 Efficiency (E)

Efficiency measures how well the parallel resources are being utilized. It is defined as the ideal parallel time divided by the actual parallel time, or equivalently, Speedup divided by the number of workers n.

$$E(n) = \frac{T_{id}(n)}{T(n)} = \frac{T_{seq}}{n \cdot T_{par}(n)} = \frac{S_p(n)}{n}$$

An ideal efficiency is 1 (or 100%), corresponding to linear speedup.

---

2. Laws of Scalability

2.1 Amdahl's Law

Amdahl's law provides a theoretical upper bound on the speedup of a program when only a fraction of it can be parallelized. If f is the strictly sequential fraction of the program (e.g., $10\% = 0.10$), the maximum

speedup is bounded by $\frac{1}{f}$ (e.g., 10×) regardless of how many cores n are added.

$$S_p(n) \leq \frac{1}{f + \frac{1-f}{n}} \xrightarrow{n \to \infty} \frac{1}{f}$$

2.2 Gustafson's Law

Gustafson's Law counteracts the pessimism of Amdahl's Law by observing that as computing power increases, the size of the problem typically increases as well. The work required for the parallel part grows much faster than the sequential part. If the sequential fraction f decreases as the data size grows, the system can achieve near-linear speedup for massive datasets.

---

3. Basic Parallel Patterns

Parallel patterns (or algorithmic skeletons) provide high-level abstractions to structure concurrent computations, avoiding low-level lock and thread management.

3.1 Pipeline

A computation is divided into k sequential stages connected by communication channels. Each stage processes a data item and passes it to the next. * **Service Time (T_s):** Bounded by the slowest stage. $T_s = \max_{i=1\ldots k}\{T_{s_i}\}$. * **Latency ($L$):** Sum of the times of all stages. $L = \sum_{i=1}^{k} T_{s_i}$. * **Optimization:** If a stage is a bottleneck, its service time can be lowered by parallelizing that specific stage internally (e.g., turning it into a Farm).

3.2 Farm (Task Farm / Master-Worker)

A Farm replicates the execution of a single function over multiple workers. It is ideal for embarrassingly parallel workloads where tasks are completely independent. * **Structure:** Consists of an *Emitter* (Scheduler) that distributes tasks, a set of *Workers* that compute the tasks independently, and a *Collector* that gathers the results. * **Service Time (T_s):** If the sequential service time is T_w and we have n_w workers, the ideal parallel service time is $T_s = \frac{T_w}{n_w}$. * **Efficiency:** A farm is highly efficient, but scheduling overhead and the sequential nature of Emitter/Collector can become bottlenecks if the communication time dominates the computation time.

---

4. Hardware & Context

When designing parallel applications, the underlying hardware heavily influences the chosen mechanism and abstraction: * **Memory Architectures:** Threads can communicate via Shared Memory (requiring synchronization primitives like locks, condition variables, or barriers) or via Message Passing (Point-to-Point or Collective communications). * **Grain Size (Granularity):** The ratio of computation time to communication time. A coarse-grained computation (high computation-to-communication ratio) is generally necessary to offset the overhead of data splitting and dispatching. * **Excess Parallelism (Oversubscription):** Spawning more parallel activities than the available hardware contexts. It allows the OS/runtime to hide latency (e.g., if one thread blocks for I/O, another can be scheduled), avoiding idle cores.

02. Structured Parallel Programming & Algorithmic Skeletons

This chapter delves into Structured Parallel Programming (SPP) and Algorithmic Skeletons. The core philosophy of this approach is the **separation of concerns**: isolating the purely sequential computational logic (business logic) from the parallel coordination and communication mechanisms.

1. The Algorithmic Skeleton Framework

Introduced by Murray Cole, **Algorithmic Skeletons** are recurring parallel paradigms abstracted into reusable, parameterizable templates. They provide high-level parallel constructs that hide the underlying complexity of thread management, synchronization, and message passing.

Skeletons are typically divided into two macro-categories based on the nature of the parallelism they exploit:

1. **Stream Parallel Skeletons:** Operate on streams of data items (tasks) flowing through the system. Examples: `pipeline`, `farm`.
2. **Data Parallel Skeletons:** Operate on large, partitioned data structures where the same computation is applied to different chunks simultaneously. Examples: `map`, `reduce`, `stencil`.

---

2. Stream Parallel Skeletons

Stream parallel skeletons model systems that process a continuous flow of input tasks. The execution is driven by the availability of new data tokens.

2.1 The Pipeline Skeleton (`pipe`)

The `pipe` skeleton connects k stages $S_1, S_2, \ldots, S_k$ in a sequence. The output stream of stage S_i becomes the input stream of stage S_{i+1}.

- **Semantics:** $\text{pipe}(S_1, S_2)(x) = S_2(S_1(x))$ extended over a stream.
- **Performance Model:**
  - **Service Time (T_s):** Bottlenecked by the slowest stage. $T_s = \max_{i=1\ldots k}\{T_{S_i}\}$
  - **Latency (L):** The sum of the times taken by all individual stages. $L = \sum_{i=1}^{k} T_{S_i}$
- **Optimization:** If a stage is identified as a bottleneck, it must be parallelized (e.g., by wrapping it in a `farm` or a nested `pipe`) to lower its service time and improve the overall throughput.

2.2 The Farm Skeleton (`farm`)

The `farm` skeleton models independent task execution (embarrassingly parallel). It consists of three entities:

1. **Emitter (E):** Fetches data from the input stream and dispatches it to the workers.
2. **Workers ($W_1 \ldots W_n$):** A pool of threads/processes computing the identical function f on different input data.
3. **Collector (C):** Gathers results from the workers and forwards them to the output stream.

- **Performance Model:**
  - Let T_E be the time the Emitter takes to schedule a task, T_W the time a worker takes to compute f, and T_C the time the Collector takes to gather a result.
  - **Service Time (T_s):** $T_s = \max\left\{T_E, \frac{T_W}{n}, T_C\right\}$
  - **Latency (L):** $L = T_E + T_W + T_C$
- **Scalability Bottleneck:** The Emitter and Collector are strictly sequential components. As the number of workers n increases, $\frac{T_W}{n}$ decreases. Eventually, T_E or T_C will become the maximum value, causing the speedup to plateau. The maximum useful number of workers is $n_{max} = \frac{T_W}{\max(T_E, T_C)}$.

---

3. Data Parallel Skeletons

Data parallel skeletons apply operations across partitions of a single, large data structure (like vectors, matrices, or graphs).

3.1 The Map Skeleton (`map`)

The `map` skeleton takes an array (or list) and applies a purely sequential function f to each element independently.

- **Semantics:** $\text{map}(f)([x_1, x_2, \ldots, x_n]) = [f(x_1), f(x_2), \ldots, f(x_n)]$
- **Execution:** The runtime partitions the array into chunks, assigns chunks to different processing elements (PEs), computes the chunks in parallel, and merges the results.
- **Completion Time (T_c):** For an array of size N distributed over n workers, the ideal completion time is $T_c = \frac{N \cdot T_f}{n} + T_{scatter} + T_{gather}$.

3.2 The Reduce Skeleton (`reduce`)

The `reduce` skeleton takes an array and aggregates its elements into a single value using a binary operator $\oplus$. For the skeleton to yield deterministic results regardless of the parallel execution tree, the operator $\oplus$ **must be associative**.

- **Semantics:** $\text{reduce}(\oplus)([x_1, x_2, \ldots, x_n]) = x_1 \oplus x_2 \oplus \cdots \oplus x_n$
- **Execution:** Typically implemented using a parallel tree reduction.
- **Completion Time (T_c):** With n workers, the reduction tree has a depth of $\log_2(n)$. Thus, parallel time scales logarithmically rather than linearly in the number of processors.

---

4. Operational Semantics for Skeletons

To formalize the behavior of these skeletons without ambiguity, we use operational semantics modeled as transition systems. States represent the current configuration of the computation, and transitions ($\rightarrow$) describe computation steps.

Let $\langle x, \tau \rangle$ represent an input stream where x is the current token and τ is the rest of the stream. Let Δ represent the inner computation of a skeleton (e.g., the worker function in a farm).

1. **Sequential execution (`seq`):**

   Applies a function f to a stream.

$$\langle \text{seq } f, \langle x, \tau \rangle \rangle \rightarrow \langle f(x) \rangle :: \langle \text{seq } f, \langle \tau \rangle \rangle$$

2. **Farm execution (`farm`):**

   The farm processes the first element of the stream and concurrently processes the rest.

$$\langle \text{farm } \Delta, \langle x, \tau \rangle \rangle \rightarrow \langle \Delta(x) \rangle \parallel \langle \text{farm } \Delta, \langle \tau \rangle \rangle$$

   (Note: The exact formalization can include explicit rules for Emitter scheduling and Collector gathering depending on the specific transition system used).

---

5. Implementation in Real Frameworks (e.g., C++ FastFlow)

In modern C++ environments, implementing these mathematical models effectively requires low-overhead runtime supports. Frameworks like FastFlow implement stream parallel skeletons using **lock-free** and **wait-free** queues (e.g., Single-Producer Single-Consumer queues for pipelines).

- In a FastFlow `ff_pipeline`, each stage is a C++ object executing a sequential loop, pulling data from a lock-free input queue and pushing to an output queue.
- In a `ff_farm`, the Emitter acts as a load balancer, distributing pointers to memory allocations rather than copying the data itself, thereby minimizing T_E and extending the scalability limits defined by Amdahl's Law.

03. Shared Memory & Low-Level Concurrency

This chapter explores the intricacies of Shared Memory architectures, the challenges of keeping memory consistent across multiple processors, and the practical implementation of concurrent execution using modern tools.

1. Shared Memory Architectures

In a shared memory system, multiple processing elements (PEs) have access to a single, global memory address space. While this simplifies the programming model (data does not need to be explicitly sent and received via messages), it introduces severe hardware and software complexities.

1.1 Cache Coherence

Modern CPUs utilize multiple levels of cache (L1, L2, L3) to hide main memory latency. If multiple cores load the same memory block into their local L1 caches and one core modifies it, the other copies become stale.

- **Coherence Protocols:** The hardware must keep all these copies consistent and coherent through specific protocols (e.g., MESI - Modified, Exclusive, Shared, Invalid).
- **Performance Impact:** When a core writes to a shared variable, the coherence protocol must invalidate or update the cached copies in other cores. This generates heavy bus traffic and latency, a phenomenon known as **False Sharing** if cores are writing to different variables that happen to reside on the same cache line.

1.2 Synchronization Primitives

To prevent race conditions when multiple threads access shared data, we must enforce mutual exclusion using synchronization mechanisms:

- **Mutexes and Locks:** Ensure that only one thread can execute a critical section at a time. However, locking forces sequential execution, impacting Amdahl's Law and adding overhead.
- **Atomics and Lock-Free Programming:** Utilizing hardware-level atomic instructions (like Compare-And-Swap) to update variables without blocking threads. This is crucial for high-performance concurrent data structures.

---

2. Branch Divergence and Predication

When executing parallel workloads on architectures with SIMT (Single Instruction, Multiple Threads) execution models, such as GPUs or highly vectorized CPU units, control flow becomes a critical bottleneck.

2.1 The Issue with Branching

If threads executing in lockstep encounter a conditional branch (e.g., an `if-else` statement) and diverge (some take the `true` path, others the `false` path), the hardware must serialize the execution. It first executes the `true` path for the active threads, masking the others, and then executes the `false` path. This halves the parallel efficiency.

2.2 Predication as a Solution

To mitigate branch divergence, one technique is **predication**. Instead of branching, the hardware computes both paths simultaneously. * **How it works:** All threads compute the instructions for both the `if` and the `else` branches. The results are guarded by a boolean predicate (flag). Only the results associated with the `true` predicate are actually committed to memory or registers. * **Trade-off:** This requires doing extra computational work, but it avoids the severe penalty of breaking the instruction pipeline and serializing the parallel execution context.

---

3. Concurrency in Modern C++

Modern standards provide robust abstractions for managing shared memory parallelism without relying on raw OS-level threads (like POSIX threads).

3.1 Standard Threads and Data Races

Managing threads directly requires explicit synchronization.

```cpp
#include <iostream>
#include <thread>
#include <vector>
#include <mutex>

std::mutex mtx;

void compute_chunk(int thread_id, int& shared_accumulator) {
    // compute local aggregate to minimize lock contention
    int local_sum = 0;
    for (int i = 0; i < 1000; ++i) {
        local_sum += (i * thread_id);
    }

    // update shared state safely
    std::lock_guard<std::mutex> lock(mtx);
    shared_accumulator += local_sum;

    std::cout << "THREAD " << thread_id << " FINISHED EXECUTION. CURRENT ACCUMULATOR: " << shared_accum
}

int main() {
    int shared_accumulator = 0;
    std::vector<std::thread> pool;

    // spawn threads
    for (int i = 0; i < 4; ++i) {
        pool.emplace_back(compute_chunk, i, std::ref(shared_accumulator));
    }
```

```
    // wait for completion
    for (auto& t : pool) {
        t.join();
    }

    std::cout << "FINAL RESULT COMPUTED SUCCESSFULLY\n";
    return 0;
}
```

3.2 OpenMP

For data-parallel loops and regular computational kernels, OpenMP provides compiler directives (`#pragma`) that dramatically simplify parallelization by handling thread creation, work distribution, and synchronization automatically.

```
#include <iostream>
#include <vector>
#include <omp.h>

void process_array(std::vector<int>& data) {
    int total_sum = 0;

    // parallelize the loop and perform a reduction on total_sum
    #pragma omp parallel for reduction(+:total_sum)
    for (size_t i = 0; i < data.size(); ++i) {
        data[i] = data[i] * 2;
        total_sum += data[i];
    }

    std::cout << "OPENMP REDUCTION COMPLETED. SUM: " << total_sum << "\n";
}
```

By leveraging these constructs, developers can bridge the gap between abstract algorithmic skeletons (like Map and Reduce) and actual high-performance execution on shared memory architectures.

04. Thread Affinity & Memory Consistency

This chapter explores the low-level interactions between the operating system, the hardware architecture, and concurrent programs. Understanding how threads are scheduled and how memory is accessed is crucial for extracting maximum performance from multi-core systems.

1. Thread and Process Affinity

When the operating system's scheduler manages a pool of threads, it is free to migrate them across different physical cores to balance the system load. While this is optimal for general-purpose computing, it introduces severe performance penalties for High-Performance Computing (HPC) and structured parallel applications.

1.1 The Cost of Thread Migration

If a thread is executing on Core C_0 and is migrated to Core C_1, the following issues arise:

- **Cache Invalidation:** The L1 and L2 caches of C_0 contain the hot data for the thread. When moved to C_1, the thread experiences a burst of cache misses, forcing the CPU to fetch data from the slower L3 cache or Main Memory.

- **Context Switch Overhead:** The registers, program counter, and translation lookaside buffer (TLB) state must be saved and restored.
- **NUMA Penalties:** In Non-Uniform Memory Access (NUMA) architectures, memory is divided into nodes assigned to specific sockets. If a thread is moved to a different socket, accessing its originally allocated memory becomes significantly slower due to inter-socket communication links (e.g., Intel UPI, AMD Infinity Fabric).

1.2 Enforcing Affinity

To prevent migration, we can explicitly bind threads to specific hardware contexts (cores or hyper-threads). This is known as **Thread Affinity**.

In Linux environments, this is often handled using the `sched_setaffinity` system call or through library wrappers. In modern C++, we access the underlying OS handle via `.native_handle()`.

2. Memory Consistency Models

Cache coherence (discussed in the previous chapter) ensures that all cores see the same value for a *single* memory location. However, it does not guarantee the order in which memory operations across *multiple* locations become visible to different cores. This is dictated by the **Memory Consistency Model**.

2.1 Sequential Consistency (SC)

The most intuitive model, proposed by Lamport. A system is sequentially consistent if the result of any execution is the same as if the operations of all processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program.

- **Drawback:** Enforcing SC completely prevents the compiler and the CPU from reordering memory instructions for optimization (e.g., storing to a write buffer while a load completes).

2.2 Relaxed Memory Models

To achieve high performance, modern hardware (like x86, and particularly ARM) reorders loads and stores.

To write correct lock-free code, developers must use **Memory Barriers** (or fences) to explicitly enforce ordering where necessary.

2.3 C++11 Memory Model

C++ provides a standard way to deal with memory ordering using `std::atomic` and `std::memory_order` flags:

- `memory_order_relaxed`: No synchronization or ordering constraints, only atomicity for the specific operation.
- `memory_order_acquire` / `memory_order_release`: A store with `release` synchronizes with a load with `acquire` on the same atomic variable. All memory writes before the release become visible to the thread doing the acquire.
- `memory_order_seq_cst`: Enforces strict sequential consistency (the default, but most expensive).

3. Mitigating False Sharing

As introduced previously, false sharing occurs when independent variables accessed by different threads reside on the same cache line (typically 64 bytes).

To solve this, we must enforce spatial separation of data in memory using alignment directives.

```cpp
#include <iostream>
#include <thread>
#include <vector>
#include <atomic>
#include <pthread.h>

// align to 64 bytes to prevent false sharing between array elements
struct alignas(64) thread_state {
    std::atomic<int> counter;
};

void compute_work(int id, thread_state* state) {
    // get native handle to pin thread to specific cpu core
    pthread_t thread = pthread_self();
    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);
    CPU_SET(id, &cpuset);

    // enforce affinity
    pthread_setaffinity_np(thread, sizeof(cpu_set_t), &cpuset);

    // execute workload using relaxed memory order for pure counters
    for(int i = 0; i < 100000; ++i) {
        state[id].counter.fetch_add(1, std::memory_order_relaxed);
    }
}

int main() {
    const int num_threads = 4;
    // allocated contiguous memory, but alignas guarantees 64-byte spacing
    thread_state states[num_threads];
    std::vector<std::thread> pool;

    for (int i = 0; i < num_threads; ++i) {
        states[i].counter.store(0, std::memory_order_relaxed);
        pool.emplace_back(compute_work, i, states);
    }

    for (auto& t : pool) {
        t.join();
    }

    return 0;
}
```

By manually orchestrating memory alignment and thread placement, the abstractions defined by parallel skeletons can be implemented with minimal hardware-level interference, achieving near-linear speedups bounded only by Amdahl's fraction.

05. Distributed Systems & Message Passing

This chapter shifts the focus from shared memory systems to distributed memory architectures, exploring how isolated processes communicate and synchronize over a network using the Message Passing paradigm, with a deep dive into the MPI standard.

1. Distributed Memory Architectures

Unlike shared memory systems where all cores access a single global address space, distributed systems consist of independent processing nodes. Each node has its own local memory, CPU, and operating system instance.

- **No Cache Coherence Issues:** Since memory is isolated, there is no hardware-level cache coherence protocol spanning across nodes. False sharing is strictly a local, intra-node issue.
- **Communication via Network:** Nodes communicate explicitly by sending and receiving messages over an interconnection network (e.g., Ethernet, InfiniBand).
- **Latency and Bandwidth:** Network communication is orders of magnitude slower than memory access. Granularity must be coarse; overlapping computation and communication becomes critical to hide network latency.

2. The Message Passing Paradigm

In the message passing model, concurrent activities (processes) interact by explicitly exchanging data payloads.

2.1 Send and Receive Primitives

The foundational operations are `send(dest, message)` and `receive(src, message)`. These primitives can be classified based on their synchronization semantics:

- **Synchronous (Blocking):** The sender blocks until the receiver has acknowledged the receipt of the message. This naturally enforces a synchronization barrier between the two processes.
- **Asynchronous (Non-Blocking):** The sender dispatches the message to a buffer (managed by the OS or network hardware) and immediately resumes execution. The receiver can fetch the message from the buffer later. This allows computation-communication overlap but requires complex buffer management.

3. Message Passing Interface (MPI)

MPI is the de-facto standard for High-Performance Computing (HPC) on distributed systems. It defines a rich API for process communication, defining a static set of processes spawned at the beginning of the execution.

3.1 Communicators and Ranks

- **Communicator (`MPI_Comm`):** A communication universe. `MPI_COMM_WORLD` is the default communicator encompassing all spawned processes.
- **Rank:** A unique integer identifier assigned to each process within a communicator, ranging from 0 to $N - 1$. Ranks are used to target destinations and identify sources.

3.2 Point-to-Point Communication

Direct communication between two specific ranks. * `MPI_Send`: Basic blocking send. Returns when the application buffer can be safely reused. * `MPI_Recv`: Basic blocking receive. Waits until a matching message arrives. * `MPI_Isend` / `MPI_Irecv`: Non-blocking variants. They return an `MPI_Request` handle that must be explicitly checked or waited upon later (using `MPI_Wait`).

3.3 Collective Communication

Operations that involve all ranks within a communicator simultaneously. They are heavily optimized for the specific network topology (e.g., using hypercube routing or tree-based reductions). * **Broadcast (`MPI_Bcast`):** Rank 0 sends the same data to all other ranks. * **Scatter (`MPI_Scatter`):** Rank 0 splits an array into chunks and sends one chunk to each rank. * **Gather (`MPI_Gather`):** All ranks send their local data chunks to Rank 0, which concatenates them. * **Reduce (`MPI_Reduce`):** All ranks provide a value, and an associative operator (like `MPI_SUM`, `MPI_MAX`) aggregates them into a single result at the root rank.

---

4. MPI Implementation in C++

Below is an implementation of a standard scatter-compute-gather pattern using MPI primitives.

```cpp
#include <iostream>
#include <vector>
#include <mpi.h>

int main(int argc, char** argv) {
    // initialize mpi environment
    MPI_Init(&argc, &argv);

    int rank;
    int size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    const int elements_per_proc = 1000;
    std::vector<int> local_data(elements_per_proc);
    std::vector<int> global_data;

    if (rank == 0) {
        std::cout << "MASTER NODE INITIALIZED WITH " << size << " PROCESSES" << std::endl;
        global_data.resize(size * elements_per_proc);

        // initialize dummy workload
        for (int i = 0; i < size * elements_per_proc; ++i) {
            global_data[i] = 1;
        }
    }

    // scatter data from root to all processes
    MPI_Scatter(global_data.data(), elements_per_proc, MPI_INT,
                local_data.data(), elements_per_proc, MPI_INT,
                0, MPI_COMM_WORLD);

    // compute local workload
    int local_sum = 0;
    for (int i = 0; i < elements_per_proc; ++i) {
        local_sum += local_data[i];
    }

    // reduce local results to global sum on root
    int global_sum = 0;
    MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        std::cout << "COMPUTATION FINISHED. GLOBAL SUM: " << global_sum << std::endl;
    }

    // tear down mpi environment
    MPI_Finalize();
    return 0;
}
```

4.1 Overlapping Computation and Communication

To squeeze maximum performance, advanced MPI code utilizes `MPI_Isend` and `MPI_Irecv`. While the network interface controller (NIC) is transferring the boundary data asynchronously, the CPU computes the inner core of the local dataset. Once the inner core is done, the code waits for the boundary transfers to complete (`MPI_Wait`) before computing the edges. This effectively hides the network latency L behind the computation time.

06. FastFlow Framework & Stream Processing

This chapter introduces FastFlow, a C++ parallel programming framework developed primarily at the University of Pisa and the University of Torino. It implements algorithmic skeletons with a strong focus on stream processing and multi-core cache-coherent architectures.

1. FastFlow Design Philosophy

FastFlow is designed to overcome the performance bottlenecks associated with traditional lock-based synchronization (like `std::mutex` or `pthread_mutex_t`). It achieves high performance through:

- **Lock-Free and Wait-Free Queues:** The core communication mechanism relies on Single-Producer Single-Consumer (SPSC) lock-free queues.
- **Zero-Copy Routing:** Instead of copying large data payloads between pipeline stages or farm workers, FastFlow streams pointers to memory allocations.
- **Cache-Conscious Behavior:** By preventing threads from blocking and keeping them pinned to specific cores, FastFlow maximizes cache hotness and minimizes context-switching overhead.

---

2. Core Abstraction: `ff_node`

The fundamental building block in FastFlow is the node. A node represents an active entity (typically mapped to an OS thread) that executes a specific sequential computation.

2.1 The `svc` Method

To define a custom computation, you create a struct or class that inherits from `ff_node_t<IN, OUT>` (or the generic `ff_node`) and overrides the `svc` (service) method.

The `svc` method takes a pointer to an input task, processes it, and returns a pointer to the output task. * Returning a valid pointer pushes the task to the output queue. * Returning `GO_ON` instructs the runtime to fetch the next task without producing output for the current cycle. * Returning `EOS` (End Of Stream) signals the termination of the stream.

---

3. Implementing Skeletons

FastFlow provides container nodes that implement the standard algorithmic skeletons.

3.1 Pipeline (`ff_Pipe`)

A pipeline connects multiple `ff_node` instances sequentially. The output queue of stage i is exactly the input queue of stage $i + 1$. Because these are strictly 1-to-1 connections, the SPSC lock-free queues operate at maximum efficiency without requiring atomic Compare-And-Swap (CAS) operations, relying only on proper memory fences.

3.2 Farm (`ff_Farm`)

A farm distributes tasks among multiple worker nodes. * **Emitter:** Pulls from a Single-Producer Single-Consumer queue and pushes to N SPSC queues (one for each worker). * **Workers:** An array of `ff_node` instances executing the same `svc` logic. * **Collector:** Reads from N SPSC queues and merges the results into a single output stream.

---

4. C++ Implementation Example

Below is a complete example of a Pipeline containing a Farm, demonstrating how to set up the nodes and handle stream tokens.

```cpp
#ifndef FASTFLOW_FARM_EXAMPLE_H
#define FASTFLOW_FARM_EXAMPLE_H

// Created by user

#include <iostream>
#include <vector>
#include <ff/ff.hpp>
#include <ff/farm.hpp>
#include <ff/pipeline.hpp>

using namespace ff;

// task definition
struct task_t {
    int id;
    double value;
};

// first stage: generates stream
struct generator : ff_node_t<task_t> {
    task_t* svc(task_t*) override {
        for (int i = 0; i < 100; ++i) {
            task_t* t = new task_t{i, i * 1.5};
            ff_send_out(t);
        }
        return EOS;
    }
};

// worker stage: processes stream in parallel
struct worker : ff_node_t<task_t> {
    task_t* svc(task_t* task) override {
        // simulate workload
        task->value = task->value * 2.0;
        return task;
    }
};

// final stage: collects results
struct collector : ff_node_t<task_t> {
    task_t* svc(task_t* task) override {
```

```
        std::cout << "PROCESSED TASK ID: " << task->id
                  << " | FINAL VALUE: " << task->value << "\n";
        delete task;
        return GO_ON;
    }
};

#endif // FASTFLOW_FARM_EXAMPLE_H

int main() {
    generator gen;
    collector col;

    // setup workers
    std::vector<std::unique_ptr<ff_node>> W;
    for(int i = 0; i < 4; ++i) {
        W.push_back(std::make_unique<worker>());
    }

    // build farm
    ff_Farm<task_t> farm(std::move(W));
    farm.remove_collector();

    // build pipeline: generator -> farm -> collector
    ff_Pipe<task_t> pipe(gen, farm, col);

    if (pipe.run_and_wait_end() < 0) {
        std::cerr << "FATAL: PIPELINE EXECUTION FAILED\n";
        return -1;
    }

    std::cout << "EXECUTION COMPLETED SUCCESSFULLY\n";
    return 0;
}
```

4.1 Memory Management Implications

Notice that the generator allocates memory on the heap (`new task_t`), and the collector is responsible for freeing it (`delete task`). Since FastFlow passes pointers, dynamic allocation is standard. However, frequent `new`/`delete` calls can cause contention on the glibc memory allocator. For extreme low-level performance, FastFlow applications often integrate custom thread-local memory allocators to bypass this OS-level bottleneck.

07. Data Parallelism & GPU Concurrency

This chapter extends the algorithmic skeleton framework to data-parallel workloads, exploring how to efficiently process large, partitioned data structures using both CPU-bound frameworks (FastFlow) and massively parallel architectures (GPUs via CUDA).

1. Data Parallelism in FastFlow

While stream parallelism processes discrete tokens over time, data parallelism operates on a finite, pre-existing data structure (like an array or matrix). FastFlow provides specific constructs to handle these workloads without the overhead of instantiating full pipeline or farm nodes for each element.

1.1 Parallel For (`ff::ParallelFor`)

The `ParallelFor` construct applies a function to a range of indices $[0, N)$. It splits the iteration space into chunks and assigns them to a pool of worker threads.

- **Chunking Strategy:** The runtime can distribute iterations statically (fixed size chunks) or dynamically (workers steal chunks when idle) to balance the load.
- **Low-Level Execution:** Unlike standard OpenMP, `ParallelFor` in FastFlow leverages the same lock-free worker pool underlying the `ff_Farm`. This avoids the OS-level thread creation and destruction overhead if multiple parallel loops are executed sequentially.

1.2 Parallel For with Reduction (`ff::ParallelForReduce`)

When the parallel loop must aggregate results into a single variable, synchronization is required to avoid data races. `ParallelForReduce` handles this by providing a thread-local accumulator and an associative reduction operator, eliminating the need for atomic instructions or mutexes during the map phase.

```cpp
#ifndef FASTFLOW_DATAPARALLEL_EXAMPLE_H
#define FASTFLOW_DATAPARALLEL_EXAMPLE_H

// Created by user

#include <iostream>
#include <vector>
#include <thread>
#include <ff/parallel_for.hpp>

using namespace ff;

void compute_data_parallel() {
    const size_t num_elements = 1000000;
    std::vector<double> data(num_elements, 2.0);
    double global_sum = 0.0;

    // initialize the parallel-for engine
    ParallelForReduce<double> pf(std::thread::hardware_concurrency());

    // execute map-reduce over the vector
    // chunking is handled automatically, using thread-local identity (0.0)
    pf.parallel_reduce(
        global_sum,
        0.0,
        0, num_elements,
        1, // step
        0, // auto chunk size
        [&](const long i, double& local_sum) {
            // map phase: square the element and accumulate locally
            local_sum += (data[i] * data[i]);
        },
        [](double& current_global, const double& local_result) {
            // reduce phase: merge local results into global
            current_global += local_result;
        }
    );

    std::cout << "PARALLEL REDUCTION COMPLETED. RESULT: " << global_sum << "\n";
```

```
}

#endif // FASTFLOW_DATAPARALLEL_EXAMPLE_H
```

---

2. GPU Concurrency & CUDA Basics

When data parallelism scales to millions of independent operations, CPU thread counts (even highly optimized ones) become a bottleneck due to context switching and complex control units. GPUs solve this using the **SIMT (Single Instruction, Multiple Threads)** execution model.

2.1 The SIMT Execution Model

In CUDA, a parallel function executed on the GPU is called a **Kernel**. When a kernel is launched, it is executed by thousands of lightweight threads.

- **Threads** are grouped into **Blocks**. Threads within the same block execute concurrently on the same Streaming Multiprocessor (SM), can synchronize easily, and share fast on-chip memory.
- **Blocks** are grouped into a **Grid**. Blocks execute independently and cannot strictly synchronize with each other during kernel execution, allowing the GPU to scale the workload across any number of available SMs.

2.2 Memory Hierarchy

Effective C++ programming on GPUs requires strict, explicit control over pointer locations and memory spaces:

- **Global Memory:** The main VRAM. It is large but extremely slow (high latency). Accessible by all threads across all blocks.
- **Shared Memory:** A small, fast, user-managed cache allocated per Block. Used to share data among threads in the same block and avoid redundant global memory fetches.
- **Registers:** The fastest memory, strictly private to each individual thread.

2.3 CUDA Kernel Execution

Writing a CUDA kernel involves defining the C++ code that a *single* thread will execute. The thread computes its physical data partition by reading hardware-provided index variables.

```
#ifndef CUDA_VECTOR_ADD_H
#define CUDA_VECTOR_ADD_H

// Created by user

#include <iostream>
#include <cuda_runtime.h>

// kernel executed by each gpu thread
__global__ void vector_add(const float* a, const float* b, float* c, int n) {
    // compute global thread index mapping physical hardware to array index
    int i = blockDim.x * blockIdx.x + threadIdx.x;

    // bound check to avoid segmentation faults
    if (i < n) {
        c[i] = a[i] + b[i];
```

```
    }
}

void run_cuda_kernel() {
    int n = 100000;
    size_t bytes = n * sizeof(float);

    float *h_a, *h_b, *h_c;
    float *d_a, *d_b, *d_c;

    // allocate host memory (initialization omitted for brevity)
    h_a = (float*)malloc(bytes);
    h_b = (float*)malloc(bytes);
    h_c = (float*)malloc(bytes);

    // allocate device memory on the vram
    cudaMalloc(&d_a, bytes);
    cudaMalloc(&d_b, bytes);
    cudaMalloc(&d_c, bytes);

    // copy data from host ram to gpu global memory over pcie
    cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice);

    // define execution configuration
    int threads_per_block = 256;
    int blocks_per_grid = (n + threads_per_block - 1) / threads_per_block;

    // launch asynchronous kernel
    vector_add<<<blocks_per_grid, threads_per_block>>>(d_a, d_b, d_c, n);

    // block cpu and copy result back to host
    cudaMemcpy(h_c, d_c, bytes, cudaMemcpyDeviceToHost);

    std::cout << "CUDA KERNEL EXECUTED SUCCESSFULLY\n";

    // free memory to prevent leaks
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    free(h_a); free(h_b); free(h_c);
}

#endif // CUDA_VECTOR_ADD_H
```

---

3. Host-Device Synchronization and Latency Hiding

A critical aspect of GPU programming is managing the asynchronous nature of kernel launches. The `<<<...>>>` syntax queues the kernel on the device and returns control to the host CPU immediately.

To measure execution time accurately or ensure data is ready before subsequent CPU operations, explicit synchronization commands like `cudaDeviceSynchronize()` must be invoked. The overhead of PCI-Express memory transfers (`cudaMemcpy`) is massive and often represents the primary bottleneck. Advanced low-level optimization requires utilizing **CUDA Streams** to asynchronously overlap host-to-device memory copies with kernel execution, effectively hiding the PCIe latency.

08. Advanced GPU Concurrency & CUDA Optimizations

Building upon the basic SIMT (Single Instruction, Multiple Threads) execution model, this chapter explores the hardware-specific details necessary to achieve peak performance on GPUs. We focus on memory access patterns, the explicit management of the memory hierarchy, and the overlapping of computation with data transfers.

1. The Warp Execution Model

While we program CUDA using a hierarchy of Grids and Blocks, the actual hardware execution on a Streaming Multiprocessor (SM) is managed in units called **Warps**. * A warp consists of 32 contiguous threads from the same block. * All threads in a warp execute the exact same instruction at the same time. * **Warp Divergence:** If threads within a warp take different control-flow paths (e.g., due to an `if-else` statement), the SM must serialize the execution of the divergent paths. It executes the `true` path while masking the inactive threads, then executes the `false` path. This drastically reduces instruction throughput.

2. Global Memory Coalescing

The bandwidth to global memory (VRAM) is a primary bottleneck in GPU computing. To maximize it, we must ensure **coalesced memory accesses**.

When threads in a warp issue a memory load or store, the hardware attempts to group (coalesce) these requests into as few memory transactions as possible (typically 32, 64, or 128 bytes). * **Coalesced Access:** Thread i accesses memory address $X + i$. The entire warp requests a contiguous block of memory, which can be fetched in a single transaction. * **Uncoalesced Access:** Threads access scattered, non-contiguous addresses. The hardware must issue multiple memory transactions, wasting bandwidth.

2.1 Data Layout: AoS vs SoA

To promote coalesced accesses, data structures should often be refactored. * **Array of Structures (AoS):** `struct { float x, y, z; } array[N];` Adjacent threads accessing `array[i].x` will fetch data spaced by the size of the struct, leading to uncoalesced accesses. * **Structure of Arrays (SoA):** `struct { float x[N], y[N], z[N]; };` Adjacent threads accessing `x[i]` will fetch contiguous memory, achieving perfect coalescing.

---

3. Shared Memory and Tiling

Shared memory is a programmable L1 cache local to each SM. It is orders of magnitude faster than global memory but limited in size (e.g., 48KB or 96KB per block).

We use shared memory to hold frequently accessed data. A common pattern is **Tiling**: 1. Threads in a block cooperatively load a "tile" of data from global memory into shared memory. 2. The block synchronizes to ensure all data is loaded. 3. Threads perform multiple computations using the fast shared memory. 4. The block synchronizes again, writes results to global memory, and moves to the next tile.

3.1 Tiling Implementation

```
#ifndef CUDA_MATRIX_MUL_TILED_H
#define CUDA_MATRIX_MUL_TILED_H

// Created by user

#include <iostream>
#include <cuda_runtime.h>
```

```cpp
#define TILE_WIDTH 16

__global__ void matrix_mul_tiled(float* d_a, float* d_b, float* d_c, int width) {
    // allocate shared memory for tiles
    __shared__ float tile_a[TILE_WIDTH][TILE_WIDTH];
    __shared__ float tile_b[TILE_WIDTH][TILE_WIDTH];

    int bx = blockIdx.x; int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;

    // identify the row and column of the output element to compute
    int row = by * TILE_WIDTH + ty;
    int col = bx * TILE_WIDTH + tx;

    float p_value = 0.0f;

    // loop over tiles
    for (int m = 0; m < width / TILE_WIDTH; ++m) {
        // load data into shared memory
        tile_a[ty][tx] = d_a[row * width + (m * TILE_WIDTH + tx)];
        tile_b[ty][tx] = d_b[(m * TILE_WIDTH + ty) * width + col];

        // sync threads to ensure tile is fully loaded
        __syncthreads();

        // compute partial dot product
        for (int k = 0; k < TILE_WIDTH; ++k) {
            p_value += tile_a[ty][k] * tile_b[k][tx];
        }

        // sync to prevent overwriting tiles in the next iteration
        __syncthreads();
    }

    d_c[row * width + col] = p_value;
}

#endif // CUDA_MATRIX_MUL_TILED_H
```

---

4. Asynchronous Execution and CUDA Streams

To fully utilize both the CPU and the GPU, as well as the PCIe bus connecting them, we must overlap data transfers with kernel execution. This is achieved using **CUDA Streams**.

A stream is a sequence of commands (memory copies, kernel launches) that execute in order. Commands in different streams can execute concurrently.

4.1 Hiding PCIe Latency

By dividing the workload into chunks and assigning each chunk to a separate stream, the hardware can perform a `cudaMemcpyAsync` for chunk $i + 1$ while the GPU is executing the kernel for chunk i.

```cpp
#ifndef CUDA_STREAMS_EXAMPLE_H
#define CUDA_STREAMS_EXAMPLE_H
```

```
// Created by user

#include <iostream>
#include <vector>
#include <cuda_runtime.h>

void run_with_streams(float* h_in, float* h_out, int n) {
    const int num_streams = 4;
    cudaStream_t streams[num_streams];

    // initialize streams
    for (int i = 0; i < num_streams; ++i) {
        cudaStreamCreate(&streams[i]);
    }

    int chunk_size = n / num_streams;
    size_t bytes = chunk_size * sizeof(float);

    float *d_in, *d_out;
    cudaMalloc(&d_in, n * sizeof(float));
    cudaMalloc(&d_out, n * sizeof(float));

    // queue operations asynchronously per stream
    for (int i = 0; i < num_streams; ++i) {
        int offset = i * chunk_size;

        // async copy host to device
        cudaMemcpyAsync(&d_in[offset], &h_in[offset], bytes, cudaMemcpyHostToDevice, streams[i]);

        // launch kernel on the specific stream
        // (assuming vector_add kernel exists)
        int threads = 256;
        int blocks = (chunk_size + threads - 1) / threads;
        // vector_add<<<blocks, threads, 0, streams[i]>>>(&d_in[offset], &d_out[offset], chunk_size);

        // async copy device to host
        cudaMemcpyAsync(&h_out[offset], &d_out[offset], bytes, cudaMemcpyDeviceToHost, streams[i]);
    }

    // wait for all streams to finish
    cudaDeviceSynchronize();

    std::cout << "ALL STREAMS EXECUTED SUCCESSFULLY. OVERLAPPING COMPLETE.\n";

    // clean up
    for (int i = 0; i < num_streams; ++i) {
        cudaStreamDestroy(streams[i]);
    }
    cudaFree(d_in);
    cudaFree(d_out);
}

#endif // CUDA_STREAMS_EXAMPLE_H
```

Note: To fully utilize `cudaMemcpyAsync`*, host memory must be page-locked (pinned memory) using* `cudaMallocHost` *rather than standard* `malloc` *or* `std::vector` *allocations.*